SC2006 Software Engineering
Lecture 14 Supplement

Conrad Watt

Lt13 brief recap

- Design patterns give you templates and intuition for solving common problems

- A design pattern is characterised by
  - its name (e.g. “the ***strategy*** pattern”)
  - a problem statement (that it can hopefully solve!)
  - a class diagram or other technical description of the work needed to implement the design pattern
  - a statement of the design pattern’s benefits and tradeoffs

Lt13 brief recap

- Design patterns are meant to document ways in which experienced coders would solve similar problems in the past

- They can help kickstart the design process, but there is a danger of being too dogmatic in following them

- Experienced programmers are trusted to come up with their own abstractions, potentially using design patterns as initial inspiration

Lt13 brief recap

- We learned about the Strategy pattern and the Observer pattern

- The strategy pattern has mostly been absorbed into modern programming languages, and opportunities to name and use it explicitly are less common
  - certain dynamic languages like JavaScript may still find value in the explicit concept

- The Observer pattern is still widely used, although we are starting to see it become more implicit in modern programming languages
  - e.g. with asynchronous computation and callbacks

Choosing when (not!) to use the factory pattern

- In a standard object-oriented language, you must choose between ordinary object initialisation and the factory pattern

  e.g.

```
new Dog(...);
new Cat(...);

vs

AnimalFactory.createDog(...);
AnimalFactory.createCat(...);
```

Choosing when (not!) to use the factory pattern

- When does one make sense over the other?

```
new Dog(...);
new Cat(...);

vs

AnimalFactory.createDog(...);
AnimalFactory.createCat(...);
```

Choosing when (not!) to use the factory pattern

- If a constructor is very complicated, but each invocation is expected to use mostly the same information

```
new A(arg1, arg2, arg3, arg4, arg5, arg6, arg7, arg8);
new B(arg1, arg2, arg3, arg4, arg5, arg6, otherArg7);

vs

new MyFactory(arg1, arg2, arg3, arg4, arg5, arg6)
myFactory.createA(arg7, arg8);
myFactory.createB(otherArg7);
```

Choosing when (not!) to use the factory pattern

- As a way to intercept object creation and do something special

```
new UniqueObj(id1);
new UniqueObj(id2);
new UniqueObj(id1); // needs to fail!

vs

MyFactory.createUniqueObj(id1);
MyFactory.createUniqueObj(id1); // returns the same obj
```

Choosing when (not!) to use the factory pattern

- As a way to group complicated internal logic used by multiple objects in one place

```
Obj1() {
  // some very complicated calculation
}

…

Obj2() {
  // the same very complicated calculation
}

vs

class MyCalcFactory …

  int doVeryComplicatedCalculation()...

  Obj1 createObj1()...
  Obj2 createObj2()...
```

Choosing when (not!) to use the factory pattern

- Your first instinct should be to keep things simple and don’t have the added complexity of the factory pattern

- There are code maintenance trade-offs

- If one of the issues above becomes pressing enough, then can consider it

- Some older libraries probably overuse the factory pattern


I had a problem so I
thought to use Java
Now I have a ProblemFactory